

CL1002 – Programming Fundamentals Lab



Lab # 03

Logical Operators, Relational and Bitwise Operators

Instructor: Muhammad Saad Rashad

Email: saad.rashad@nu.edu.pk

Department of Computer Science,
National University of Computer and Emerging Sciences FAST
Peshawar

1. Relational Operators:

Relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0

Operator	Meaning of Operator	Example
==	Equal to	5 == 3 is evaluated to 0
>	Greater than	5 > 3 is evaluated to 1
<	Less than	5 < 3 is evaluated to 0
!=	Not equal to	5 != 3 is evaluated to 1
>=	Greater than or equal to	5 >= 3 is evaluated to 1
<=	Less than or equal to	5 <= 3 is evaluated to 0

2. Logical Operators:

In C programming language, a logical operator is a symbol that is used to combine two or more conditions to produce a logical result. The result of a logical operation is a boolean value (either true or false), which indicates whether the combined conditions are satisfied or not.

There are three logical operators in C:

1. Logical AND (&&)
2. Logical OR (||)
3. Logical NOT (!)

The logical operators are commonly used in decision-making statements (such as if-else statements and while loops) to test multiple conditions and execute different code blocks based on the result of the tests.

1. Logical AND (&&):

This type of operator returns true when both the conditions that are under consideration happen to be satisfied. In any other case, it is bound to return false.

Examples:

```

#include<stdio.h>
int main()
{
    int a =10;
    int b=20;
    int check_num;

    check_num =(a>0 && b>0);

    printf("Both numbers are positive: %d",check_num);

    return 0;
}

```

2. Logical OR (||):

This type of operator returns true even when both or even one of the conditions that are under consideration are satisfied. In any other case, it is bound to return false.

Example:

```

#include<stdio.h>
int main()
{
    int a =10;
    int b=20;
    int check_zero;

    check_zero =(a==0 || b==0);

    printf("one of the numbers is atleast zero: %d",check_zero);

    return 0;
}

```

3. Logical NOT (!):

This type of operator returns true whenever the conditions that are under condition are not at all satisfied.

```
#include<stdio.h>
int main()
{
    int is_raining;
    printf("Enter the value 1 if it is raining and 0 otherwise: ");
    scanf("%d",&is_raining);

    int not_raining = !is_raining;

    printf("Rain status is 1 incase of no rain and 0 otherwise: %d\n", not_raining);
}
```

3. Bitwise Operators:

Bitwise operators are a type of operator in C that allow you to manipulate individual bits within an integer or character variable. There are six bitwise operators in C:

1. Bitwise AND (&)
2. Bitwise OR (|)
3. Bitwise XOR (^)
4. Bitwise NOT (~)
5. Left shift (<<)
6. Right shift (>>)

Here's a brief description of each bitwise operator:

1. **Bitwise AND (&):** This operator performs a bitwise AND operation on two integers or characters. Each bit of the output is 1 only if the corresponding bits of both inputs are 1.

Example:

```
#include<stdio.h>
int main()
{
    int a = 5; // binary: 101
    int b = 3; // binary: 011
    int c = a & b; // binary: 001

    printf("a & b = %d\n", c);
}
```

In this code snippet, we define two integer variables *a* and *b*, and initialize them to the decimal values 5 and 3, respectively. We then use the bitwise AND operator (&) to perform a bitwise AND operation on *a* and *b*, and store the result in the variable *c*. The binary representations of *a*, *b*, and *c* are also shown as comments in the code.

The bitwise AND operator compares the bits of *a* and *b* at each position, and outputs a 1 only if both bits are 1. In this example, the bitwise AND of *a* and *b* results in the binary number 001, which is equal to the decimal value 1. The `printf()` function is used to display the value of *c* on the console.

2. **Bitwise OR (|):** This operator performs a bitwise OR operation on two integers or characters. Each bit of the output is 1 if either of the corresponding bits of the inputs is 1.

Example:

```

#include<stdio.h>
int main()
{
    int a = 5; // binary: 101
    int b = 3; // binary: 011
    int c = a | b; // binary: 111

    printf("a | b = %d\n", c);
}

```

In this code snippet, we define two integer variables `a` and `b`, and initialize them to the decimal values 5 and 3, respectively. We then use the bitwise OR operator (`|`) to perform a bitwise OR operation on `a` and `b`, and store the result in the variable `c`. The binary representations of `a`, `b`, and `c` are also shown as comments in the code.

The bitwise OR operator compares the bits of `a` and `b` at each position, and outputs a 1 if either or both bits are 1. In this example, the bitwise OR of `a` and `b` results in the binary number 111, which is equal to the decimal value 7. The `printf()` function is used to display the value of `c` on the console.

3. **Bitwise XOR (^):** This operator performs a bitwise XOR (exclusive OR) operation on two integers or characters. Each bit of the output is 1 if the corresponding bits of the inputs are different (i.e., one is 0 and the other is 1).

Example:

```

#include<stdio.h>
int main()
{
    int a = 5; // binary: 101
    int b = 3; // binary: 011
    int c = a ^ b; // binary: 110

    printf("a ^ b = %d\n", c);
}

```

In this code snippet, we define two integer variables `a` and `b`, and initialize them to the decimal values 5 and 3, respectively. We then use the bitwise XOR operator (^) to perform a bitwise XOR (exclusive OR) operation on `a` and `b`, and store the result in the variable `c`. The binary representations of `a`, `b`, and `c` are also shown as comments in the code.

The bitwise XOR operator compares the bits of `a` and `b` at each position, and outputs a 1 only if the two bits are different (i.e., one is 0 and the other is 1). In this example, the bitwise XOR of `a` and `b` results in the binary number 110, which is equal to the decimal value 6. The `printf()` function is used to display the value of `c` on the console.

4. **Bitwise NOT (~):** This operator performs a bitwise NOT operation on an integer or character. It inverts all the bits of the input (i.e., changes all the 0s to 1s and all the 1s to 0s).

Example:

```

#include<stdio.h>
int main()
{
    int a = 5; // binary: 0000 0101
    int b = ~a; // binary: 1111 1010 (two's complement)

    printf("~a = %d\n", b);
}

```

5. **Left shift (<<):** This operator shifts the bits of an integer or character to the left by a specified number of bits. It fills the empty bits on the right with 0s.

Example:

```

#include<stdio.h>
int main()
{
    int a = 5; // binary: 0000 0101
    int b = a << 2; // binary: 0001 0100

    printf("a << 2 = %d\n", b);
}

```

In this code snippet, we define an integer variable `a` and initialize it to the decimal value 5. We then use the left shift operator (`<<`) to perform a left shift operation on `a`, shifting its binary representation to the left by 2 positions, and store the result in the variable `b`.

The left shift operator shifts each bit of the operand to the left by the specified number of positions, and pads the right-hand side with zeros. In this example, the binary representation of `a` is 0000 0101, and the left shift of `a` by 2 positions results in the binary number 0001 0100, which is equal to the decimal value 20. The `printf()` function is used to display the value of `b` on the console.

6. **Right shift (>>):** This operator shifts the bits of an integer or character to the right by a specified number of bits. It fills the empty bits on the left with the most significant bit (i.e., the sign bit for signed integers).

Example:

In this code snippet, we define an integer variable `a` and initialize it to the decimal value 20. We then use the right shift operator (`>>`) to perform a right shift operation on `a`, shifting its binary representation to the right by 2 positions, and store the result in the variable `b`.

The right shift operator shifts each bit of the operand to the right by the specified number of positions, and pads the left-hand side with zeros. In this example, the binary representation of `a` is 0001 0100, and the right shift of `a` by 2 positions results in the binary number 0000 0101, which is equal to the decimal value 5. The `printf()` function is used to display the value of `b` on the console.

Bitwise operators are useful in many applications, such as bit manipulation, data compression, and encryption.

References:

1. <https://byjus.com/gate/logical-operators-in-c/>
2. <https://www.programiz.com/c-programming/c-operators>

Note:

- Copied tasks will be awarded **zero** marks without any investigation.
- Comments your code properly.
- Note that these tasks marks could be graded through a viva in the lab.
- Please submit your file in this format (roll-no-name) i.e (22P-9111-Saad.c).

4. Lab Tasks:

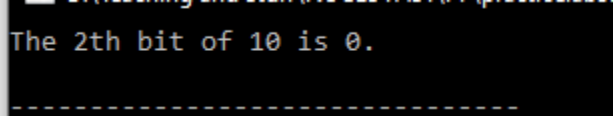
1. Take two inputs from user and check if either one of the inputs is odd print `check_odd` to 1 otherwise 0 (Don't use if else statement)
2. Take two inputs from users and check for the age, if the age is greater than 20 and less than 25 than print the status 1 otherwise 0 (Don't use if else statement)

Hint: (Check_age: 1 in case the age is greater than 20 and less than 25 otherwise 0)

3. Take a number from the user and extract the nth bit of a number (take both number and nth bit value from the user)

Hint: (Define an integer variable num and initialize it to the decimal value taken from user e.g 10, which is equivalent to the binary number 0000 1010. Define an integer variable n and initialize it to the value taken from user e.g 2, indicating that we want to extract the 2nd bit of num.

To extract the 2nd bit of num, we first create a mask by left-shifting the value 1 by n positions, resulting in the binary number 0000 0100. Then perform a bitwise AND of num and the mask to extract the nth bit of num. Finally, right-shift the result by n positions to obtain the value of the nth bit. In this case, the result of the bitwise AND is 0000 0000, so the value of the 2nd bit of num is 0.)



```
The 2th bit of 10 is 0.
```

4. Define two RGB colors as combinations of red, green, and blue values. Use unsigned char variables to represent each color component, since they can only take values between 0 and 255. Use the bitwise OR operator (|) to combine the two colors into a single color.

To combine two RGB colors, First convert each color to a 24-bit unsigned integer by left-shifting the red value by 16 positions, the green value by 8 positions, and the blue value by 0 positions (since it is already in the right position). Then perform a bitwise OR of the two colors to combine them into a single 24-bit color value.

Hint:

```
unsigned char red1 = 0xFF;    // 255 (red)
unsigned char green1 = 0x00;  // 0 (green)
unsigned char blue1 = 0x00;   // 0 (blue)

unsigned char red2 = 0x00;    // 0 (red)
unsigned char green2 = 0xFF;  // 255 (green)
unsigned char blue2 = 0x00;   // 0 (blue)
```